

ksqlDB - I

Stream processing Made Simple with Kafka

ksqlDB

The database purpose-built for stream processing applications.

build stream processing applications on top of Apache Kafka with the ease of building traditional applications on a relational database using SQL



Easily Build Real-Time Apps

ksqlDB at a Glance

ksqlDB is a database for building real-time applications that leverage stream processing

Joins

Aggregates

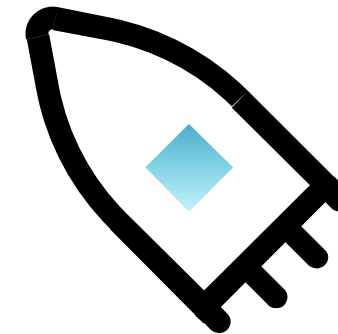
Push & Pull Queries

Filters

User-Defined
Functions

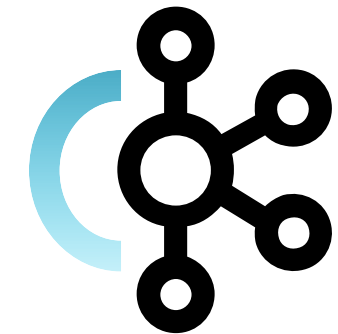
Connectors

ksqlDB



Compute

Apache Kafka®



Storage



```
CREATE TABLE activePromotions AS
SELECT rideId,
       qualifyPromotion(distanceToDst) AS promotion
FROM locations
GROUP BY rideId
EMIT CHANGES
```

Build a complete real-time application with
just a few SQL statements

Use One Solution with a Simple SQL Syntax

Use one, lightweight SQL syntax to build complete real-time applications and streaming ETL setups

```
CREATE STREAM payments
  (user VARCHAR, payment_amount
  INT)
WITH
  (kafka_topic = 'all_payments',
   key         = 'user',
   value_format = 'avro');
```

Enrich data in motion with a cloud-native stream processing solution

USER	Payment
Jay	\$10
Sue	\$15
Fred	\$5
...	...

Create aggregations that can serve push and pull queries to applications and microservices

USER	Credit Score
Jay	660
Sue	710
Fred	595
Fred	595
Fred	595

Stream Processing in Action - KSQL

data stream emitted by a production line

```
{  
  "reading_ts": "2021-06-24T09:30:00-05:00",  
  "sensor_id": "aa-101",  
  "production_line": "w01",  
  "widget_type": "acme94",  
  "temp_celcius": 23,  
  "widget_weight_g": 100  
}
```

- Alert if the line produces an item that is over a threshold weight
- Monitor the rate of item production
- Detect anomalies in the equipment
- Store the data for analytics dashboards and ad hoc querying

```
SELECT *  
FROM WIDGETS  
WHERE WEIGHT_G > 120
```

```
SELECT COUNT(*)  
FROM WIDGETS  
WINDOW TUMBLING (SIZE 1 HOUR)  
GROUP BY PRODUCTION_LINE
```

```
SELECT AVG(TEMP_CELCIUS) AS TEMP  
FROM WIDGETS  
WINDOW TUMBLING (SIZE 5 MINUTES)  
GROUP BY SENSOR_ID  
HAVING TEMP > 20
```

```
CREATE SINK CONNECTOR dw WITH (  
  connector.class = S3Connector,  
  topics = widgets  
  [...]  
);
```

Relationship Between Streams and Tables

ksqlDB Stream

Person	Location
Robin	Leeds

Unbounded Stream of Events

ksqlDB Table

Person	Location
Robin	Leeds

Current State for a Given Key

Relationship Between Streams and Tables

ksqlDB Stream

Person	Location
Robin	Leeds
Robin	London

Unbounded Stream of Events

ksqlDB Table

Person	Location
Robin	London

Current State for a Given Key

Relationship Between Streams and Tables

ksqlDB Stream



Person	Location
Robin	Leeds
Robin	London
Allison	Denver

Unbounded Stream of Events

ksqlDB Table

Person	Location
Robin	London
Allison	Denver

Current State for a Given Key

Relationship Between Streams and Tables

ksqlDB Stream



Person	Location
Robin	Leeds
Robin	London
Allison	Denver
Allison	Boulder

Unbounded Stream of Events

ksqlDB Table

Person	Location
Robin	London
Allison	Boulder

Current State for a Given Key

Relationship Between Streams and Tables

ksqlDB Stream

Person	Location
Robin	Leeds
Robin	London
Allison	Denver
Allison	Boulder
Robin	Ilkley

Unbounded Stream of Events

ksqlDB Table

Person	Location
Robin	Ilkley
Allison	Boulder

Current State for a Given Key

Relationship Between Streams and Tables

ksqlDB Stream

Person	Location
Robin	London
Allison	Denver
Allison	Boulder
Robin	Ilkley
Allison	Vail

Unbounded Stream of Events

ksqlDB Table

Person	Location
Robin	Ilkley
Allison	Vail

Current State for a Given Key

Stream V Table

Sequence of events

<K1, V1>
<K1, V2>
<K1, V3>
<K2, V1>

Stream

<K1, V1>
<K1, V2>
<K1, V3>
<K2, V1>

Table

<K1, V3>
<K2, V1>

A stream models the full history of events, while a table captures the latest state of each unique key. The stream and table representations of the preceding sequence are shown above.

They can either be created directly on top of Kafka topics (we refer to these as source collections) or derived from other streams and tables (referred to as derived collections).

Interacting with ksqlDB

Command Line Interface (CLI)



Copyright 2017-2021 Confluent Inc.

CLI v0.18.0, Server v0.18.0 located at http://ksqldb:8088
Server Status: RUNNING

Having trouble? Type 'help' (case-insensitive) for a rundown of how things work!

ksql> █

Web UI

Editor Flow Streams Tables Running queries

```
1 CREATE OR REPLACE STREAM SHIP_STATUS_REPORTS WITH (KAFKA_TOPIC='SHIP_STATUS_REPORTS', PAR
2 STATUS_REPORT.ROWTIME STATUS_TIMESTAMP,
3 STATUS_REPORT.*,
4 SHIP_INFO.*,
5 STRUCT('lat':=STATUS_REPORT.LAT, 'lon':=STATUS_REPORT.LON) LOCATION
6 FROM AIS_MSG_TYPE_1_2_3 STATUS_REPORT
7 LEFT OUTER JOIN SHIP_INFO SHIP_INFO ON ((STATUS_REPORT.MMSI = SHIP_INFO.MMSI))
8 EMIT CHANGES;
```

● [Add query properties](#)

Stop

Run query

REST API



Collections

In ksqlDB, we don't query Kafka topics directly. We query collections (i.e., streams and tables).

```
CREATE [ OR REPLACE ] { STREAM | TABLE } [ IF NOT EXISTS ] <identifier> (  
    column_name data_type [, ... ]  
) WITH (  
    property=value [, ... ]  
)
```

Using ksqlDB

create a stream called MOVEMENTS

```
CREATE STREAM MOVEMENTS (PERSON VARCHAR KEY, LOCATION VARCHAR) WITH  
(VALUE_FORMAT='JSON', PARTITIONS=1, KAFKA_TOPIC='movements');
```

```
INSERT INTO MOVEMENTS VALUES ('Allison', 'Denver');  
INSERT INTO MOVEMENTS VALUES ('Robin', 'Leeds');  
INSERT INTO MOVEMENTS VALUES ('Robin', 'Ilkley');  
INSERT INTO MOVEMENTS VALUES ('Allison', 'Boulder');
```

Schema Registry already stores field names and types, so specifying the data types again in a statement is redundant (ksqlDB can just pull the schema information from CREATE Schema Registry)

The exception to this is when you need to specify a key column. In ksqlDB, you can use the PRIMARY KEY (for tables) or KEY (for streams) identifier to tell ksqlDB which column to read for the message key.

Using KSQLDB

specify a partial schema that contains the PRIMARY KEY column, which tells ksqlDB to read this column for the message key

Explicit types	Inferred types ^a
<pre>CREATE TABLE titles (id INT PRIMARY KEY, title VARCHAR) WITH (KAFKA_TOPIC='titles', VALUE_FORMAT='AVRO', PARTITIONS=4);</pre>	<pre>CREATE TABLE titles (id INT PRIMARY KEY) WITH (KAFKA_TOPIC='titles', VALUE_FORMAT='AVRO', PARTITIONS=4);</pre>

If our titles data was instead formatted as AVRO, and the related Avro schema was stored in Schema Registry, we could use either of the above CREATE statements to create our titles table

Using ksqlDB

```
#show streams;
```

```
#SET 'auto.offset.reset' = 'earliest';
```

(to see all of the data that's already in the stream, and not just new messages as they arrive)

Now run the following SELECT to show all of the events in MOVEMENTS:

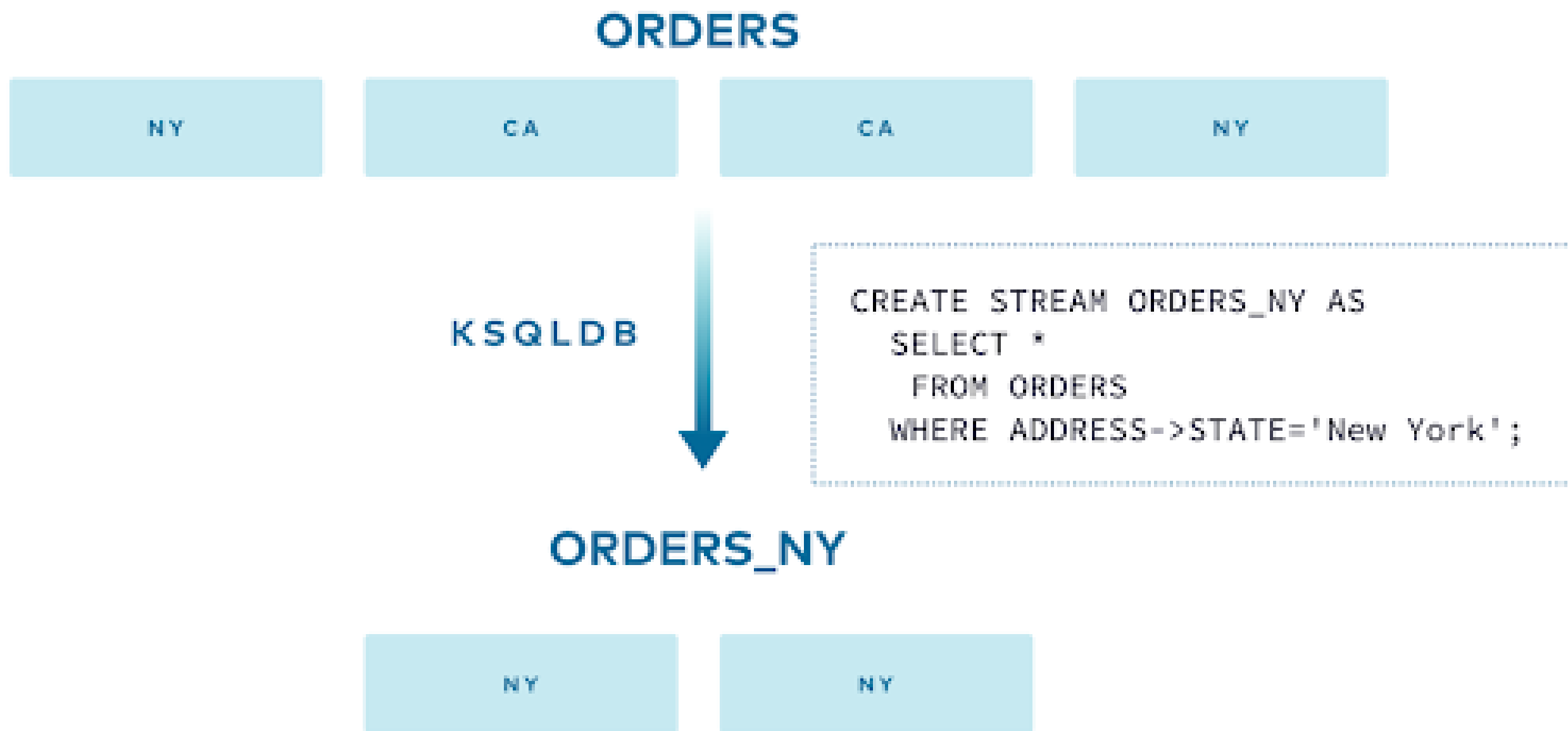
```
#SELECT * FROM MOVEMENTS EMIT CHANGES;
```

Query

```
SELECT select_expr [, ...]  
FROM from_item  
[ LEFT JOIN join_collection ON join_criteria ]  
[ WINDOW window_expression ]  
[ WHERE condition ]  
[ GROUP BY grouping_expression ]  
[ PARTITION BY partitioning_expression ]  
[ HAVING having_expression ]  
EMIT CHANGES  
[ LIMIT count ];
```

Transient push queries like the preceding one will not survive restarts of your ksqlDB server.

Filtering with ksqlDB



ksqlDB streams are backed by Kafka topics, this means that you are writing this data Directly to a Kafka topic:

```
ksql> SHOW TOPICS;
```

Kafka Topic	Partitions	Partition Replicas
ORDERS_NY	6	1
orders	6	1

```
ksql> PRINT ORDERS_NY LIMIT 2;
Key format: ^\_(\ツ)_/^- no data processed
Value format: AVRO
rowtime: 2021/02/22 11:16:10.881 Z, key: <null>, value: {"ORDERTIME":1613854570,"ORDERID":1,"ADDRESS":{"STATE":"New York","CITY":"New York","COUNTRY":"USA"},"ITEMS":[{"ITEMID":1,"ITEMNAME":"ITEM1"}]}
rowtime: 2021/02/22 10:57:36.879 Z, key: <null>, value: {"ORDERTIME":1613854056,"ORDERID":1,"ADDRESS":{"STATE":"New York","CITY":"New York","COUNTRY":"USA"},"ITEMS":[{"ITEMID":1,"ITEMNAME":"ITEM1"}]}
Topic printing ceased
ksql>
```

Lab :

Install KSQL DB – 60 Minutes(D)

Workflow using KSQL - CLI – 90 Minutes